04-Express-API/05-tools-best-practices.md

🔧 Tools & Best Practices

🎯 Professional Development Setup

Essential Tools & Techniques

Optimize your Express development workflow

🔄 nodemon - Auto-Restart Server

The Problem

Every time you change code, you must:

1. Stop the server (CTRL+C)
2. Restart: `node index.js`

This gets tedious quickly! ⏰

The Solution: nodemon

Automatically restarts your server when files change!

Installation

```
npm i -g nodemon
```

Global installation because it's a dev tool.

🚀 Using nodemon

Start Your Server

Instead of:

```
node index.js
```

Use:

```
nodemon index.js
```

Output:

```
[nodemon] 2.0.7
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Listening on port 3000...
```

Auto-Restart

When you save a file:

```
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Listening on port 3000...
```

✅ No more manual restarts!

🌍 Environment Variables

Don't Hardcode Configuration!

❌ Bad Practice:

```
app.listen(3000, () => {
    console.log('Listening on port 3000...');
});
```

✅ Better:

```
const port = process.env.PORT || 3000;
app.listen(port, () => {
    console.log(`Listening on port ${port}...`);
});
```

Why?

- Different environments use different ports
- Production servers set `PORT` variable
- Flexible configuration
- Deployment ready

🔧 Setting Environment Variables

On Different Platforms

Mac/Linux:

```
export PORT=5000
nodemon index.js
```

Windows CMD:

```
set PORT=5000
nodemon index.js
```

Windows PowerShell:

```
$Env:PORT = 5000
nodemon index.js
```

Result

```
Listening on port 5000...
```

🔒 Environment Variables Best Practices

Important Notes

⚠️ Environment variables are bound to a shell session!

- A variable in one terminal won't exist in another
- Best practice: Set in VS Code's integrated terminal
- For persistent vars: Use `.bashrc`, `.zshrc`, or system settings

Production Setup

Create a `.env` file (not committed to git!):

```
PORT=5000
DB_CONNECTION=mongodb://localhost/mydb
API_KEY=your-secret-key
```

Use a package like `dotenv`:

```
npm i dotenv
```

```
require('dotenv').config();
const port = process.env.PORT || 3000;
```

🧪 Vivesbib API Lab

Build Your First Real API

Assignment: Create an endpoint for book genres

URL: `https://bib.vivesapp.be/api/genres`

Your Task:

1. Create a new Express project
2. Set up routes for book genres
3. Implement GET, POST, PUT, DELETE
4. Add input validation
5. Test with Postman

See: GitHub Classroom - Lecture 4
Link: On Toledo

📋 API Development Checklist

Before You Start

- ☐ Install Express and Joi
- ☐ Install nodemon globally
- ☐ Install Postman or REST Client extension
- ☐ Create package.json
- ☐ Set up .gitignore

For Each Endpoint

- ☐ Define the route
- ☐ Handle request parameters
- ☐ Validate input (if needed)
- ☐ Return appropriate status codes
- ☐ Test with Postman
- ☐ Handle error cases

💡 Express Best Practices

Code Organization

✅ **DO:** Use middleware for common tasks
✅ **DO:** Extract validation functions
✅ **DO:** Use environment variables
✅ **DO:** Implement proper error handling
✅ **DO:** Return consistent response formats

❌ **DON'T:** Hardcode configuration
❌ **DON'T:** Skip input validation
❌ **DON'T:** Use wrong HTTP status codes
❌ **DON'T:** Expose sensitive error details
❌ **DON'T:** Repeat validation logic

🎨 Response Format Standards

Consistent API Responses

Success Response:

```
{
    "success": true,
    "data": {
        "id": 1,
        "name": "course1"
    }
}
```

Error Response:

```
{
    "success": false,
```

```
    "error": {
        "message": "Course not found",
        "code": 404
    }
}
```

Benefits

- Predictable structure
- Easier for clients to parse
- Better error handling
- Professional API

📊 HTTP Status Codes Reference

Code	Meaning	When to Use
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST
400	Bad Request	Validation failed
404	Not Found	Resource doesn't exist
500	Server Error	Something went wrong

🎯 Chapter 4 Complete!

What You've Learned

✅ RESTful API principles
✅ Express.js framework
✅ CRUD operations (GET, POST, PUT, DELETE)
✅ Route parameters and query strings

✅ Input validation with Joi
✅ Using Postman and nodemon
✅ Environment variables

Next: Learn about middleware in Express!

📝 Assignment

GitHub Classroom Lab

Complete the Chapter 4 lab assignment:

🔗 **See Toledo for the GitHub Classroom link (Lecture 4)**

Build the Vivesbib API with book genres!

🏠 Course Home | 📘 Chapter 4 Home

← Previous: PUT & DELETE | Next Chapter: Middleware →